

Aplicación Web Responsiva con JHipster.

Profesor: Ph.D Diego José Luis Botia Valderrama.

Objetivo General:

Comprender el desarrollo de una aplicación web responsiva con el framework JHipster y cómo puede generar aplicaciones completas y rápidas generando productividad al desarrollador.

Objetivos Específicos:

- Configurar el entorno de desarrollo con JHipster.
- Realizar una aplicación web completa mostrando la integración de las herramientas para generar aplicaciones modernas.
- Diseñar el modelo de dominio de la aplicación por medio del lenguaje JDL.
- Comprender el funcionamiento de los principales componentes de JHipster.
- Realizar un proceso de despliegue a un ambiente de producción.

Herramientas Empleadas:

- Node JS y NPM
- JHipster
- GIT
- MySQL
- JavaEE 7 u 8
- JDK 11 o 17
- REACT JS
- Spring Boot
- Docker Desktop
- Gatling

Introducción

JHipster es uno de los proyectos de Software Libre más interesantes en la actualidad, debido a que permite crear aplicaciones modernas de una forma rápida y sencilla combinando frameworks tan conocidos como Bootstrap, React / Angular / VUE y Spring Boot. Gracias a estos frameworks se integra la robustez de un backend con JavaEE y NodeJS con la extensibilidad generada en el frontend con herramientas como bower, gulp y grunt.

JHipster es un proyecto creado en Octubre de 2013 por Julien Dubois. Actualmente cuenta con un núcleo de 16 desarrolladores y cerca de 280 contribuyentes. El sitio oficial del proyecto es: <http://jhipster.github.io>. Este proyecto está escrito principalmente en JavaScript y Java.

El principal núcleo de este framework es Yeoman, el cual es un generador de código completo de tipo scaffolding. Gracias a esta herramienta se pueden generar aplicaciones web responsivas en solo minutos.

Arquitectura

JHipster es un stack completo de herramientas integradas entre las que se destacan las siguientes:

Lado cliente:

Modelo SPA (Single page application)

- Responsive Web Design
- HTML5 Boilerplate
- Twitter Bootstrap

Fundamentos Sistemas de Información
Universidad de Antioquia

- REACTJS / AngularJS / VUEJS
- Compatible con navegadores modernos
- Soporte completo para internacionalización i18n
- Soporte opcional Sass para diseño CSS
- Soporte opcional WebSocket con Spring Websocket

Con el framework de yeoman:

- Fácil instalación de nuevas librerías JavaScript con Bower
- Construcción, optimización y recarga en tiempo real con Gulp.js y Webpack
- Testing con Karma y PhantomJS

También tiene soporte para el motor de plantillas Thymeleaf, para generar páginas web en el lado del servidor

Lado del servidor:

- Spring Boot para la configuración de la aplicación de forma sencilla
- Configuración con Maven o Gradle para la construcción, test y lanzamiento de la aplicación con perfiles de desarrollo y producción.
- Spring Security
- Spring MVC REST + Jackson
- Soporte opcional WebSocket con Spring Websocket
- Spring Data JPA + Bean Validation
- Actualizaciones de base de datos con Liquibase
- Integración con Docker
- Soporte de Cacheo de segundo nivel con Hibernate
- Lenguaje de generación de entidades de datos con JHL
- Soporte para MongoDB si deseas usar una base de datos NoSQL orientada a documentos en vez de JPA.
- Soporte para Cassandra si deseas usar una base de datos NoSQL orientada a columnas en vez de JPA.

Preparado para ir a producción:

- Monitorización con Metrics
- Cacheo con ehcache (cache local) o hazelcast (cache distribuido)
- Clusterización de sesiones HTTP opcional con hazelcast
- Recursos estáticos optimizados (filtro gzip, HTTP headers cacheados)
- Gestión de logs con Logstash, configurable en tiempo de ejecución
- Pooling de conexiones con HikariCP para un rendimiento óptimo
- Creación de un WAR estándar o un JAR ejecutable

Procedimiento

Instalación del ambiente

1. Verifique que se encuentre instalado JAVA 11 o 17, si no lo tiene puede descargar de la página oficial de Oracle acá: <http://www.oracle.com/technetwork/java/javase/downloads/index.html>
2. Instale Apache Maven (preferiblemente) (<http://maven.apache.org/>) java o Gradle (<https://gradle.org/>)
3. Instale la herramienta GIT ya que acá se gestionará el versionamiento del proyecto. Puede descargarlo de <https://git-scm.com/downloads>
4. Instale NodeJS desde la siguiente página: <https://nodejs.org/en/>
Nota: Si ya tiene instalado NodeJS es recomendable actualizarlo desde una terminal o consola usando el comando **npm install -g npm**
5. Para instalar JHipster debe colocar el siguiente comando **npm install -g generator-jhipster**
6. Nota: Si usa Windows es recomendable instalar el paquete phantomJS con el comando **npm install -g phantomjs**

prebuilt. Debido a algunos bugs detectados en algunas versiones de Windows es necesario hacer este paso. Ver:

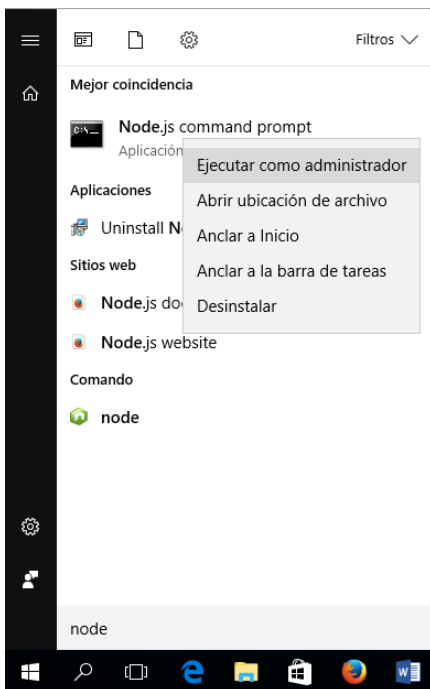
<https://github.com/Medium/phantomjs/issues/659>

```
C:\Users\Usuario\Documents\NetBeansProjects\bog1>npm install -g phantomjs-prebuilt
C:\Users\Usuario\AppData\Roaming\npm\phantomjs -> C:\Users\Usuario\AppData\Roaming\npm\node_modules\phantomjs-prebuilt\bin\phantomjs

> phantomjs-prebuilt@2.1.14 install C:\Users\Usuario\AppData\Roaming\npm\node_modules\phantomjs-prebuilt
> node install.js

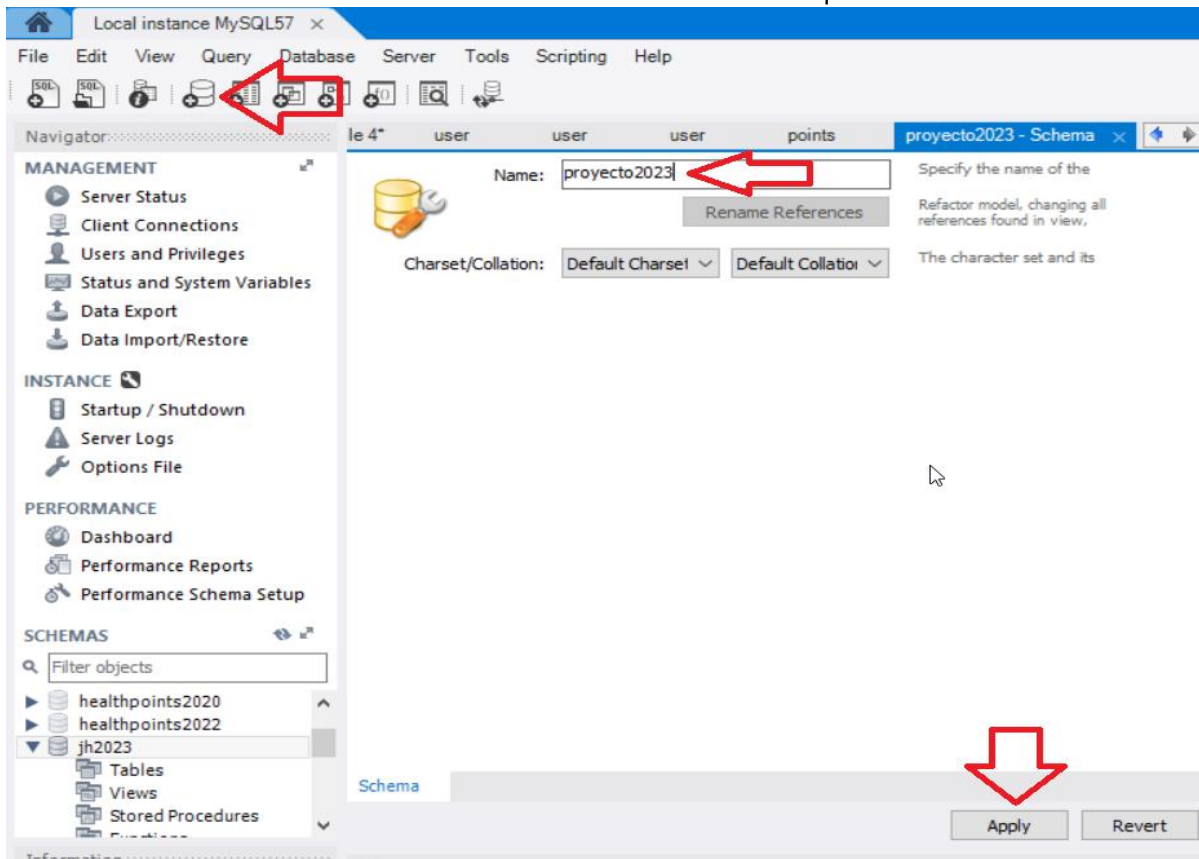
Considering PhantomJS found at C:\Users\Usuario\AppData\Roaming\npm\phantomjs.CMD
Looks like an "npm install -g" on windows; skipping installed version.
Download already available at C:\Users\Usuario\AppData\Local\Temp\phantomjs\phantomjs-2.1.1-windows.zip
Verified checksum of previously downloaded file
Extracting zip contents
Removing C:\Users\Usuario\AppData\Roaming\npm\node_modules\phantomjs-prebuilt\lib\phantom
Copying extracted folder C:\Users\Usuario\AppData\Local\Temp\phantomjs\phantomjs-2.1.1-windows.zip-extract-1491869902431\phantomjs-2.1.1-windows
Data\Roaming\npm\node_modules\phantomjs-prebuilt\lib\phantom
Writing location.js file
Done. Phantomjs binary available at C:\Users\Usuario\AppData\Roaming\npm\node_modules\phantomjs-prebuilt\lib\phantom\bin\phantomjs.exe
- balanced-match@0.4.2 node_modules\phantomjs-prebuilt\node_modules\balanced-match
- bl@1.1.2 node_modules\phantomjs-prebuilt\node_modules\bl
- concat-map@0.0.1 node_modules\phantomjs-prebuilt\node_modules\concat-map
- brace-expansion@1.1.6 node_modules\phantomjs-prebuilt\node_modules\brace-expansion
- fs.realpath@1.0.0 node_modules\phantomjs-prebuilt\node_modules\fs.realpath
```

Nota: En Windows se recomienda ejecutar la consola de NodeJS como Administrador. Si usa alguna “distro” de GNU/Linux debe ejecutar desde la terminal los comandos con **sudo**.



Creación de la base de datos

Para este ejercicio usaremos la Base de datos MySQL, por lo tanto desde MySQL Workbench o desde una consola cree un nuevo esquema llamado **proyecto2026** y luego haga click en el botón Apply como se observa a continuación:



Nota: Para este ejemplo el usuario y password de la base de datos será **root**. Si lo desea puede usar un usuario y password diferente.

Crear un nuevo proyecto con JHipster

Para crear un nuevo proyecto, se crea previamente una nueva carpeta llamada **proyecto2026** dentro del directorio de trabajo que desee en Windows o en una terminal Linux use el comando **mkdir proyecto2026**.

Ahora desde una consola de NodeJS, dentro de la ruta del directorio de trabajo coloque el comando **jhipster**, después de unos minutos se realizarán una serie de preguntas relacionadas con el tipo de aplicación, framework de frontend a usar, tipo de base de datos, soporte a internacionalización, gestor de dependencias, entre otros.

Nota: Si observa en la pregunta sobre el tipo de aplicación a parte del soporte a creación de aplicaciones monolíticas es posible crear aplicaciones basadas en Microservicios con contenedores conocidos como Docker o Kubernetes.

A continuación, se presenta la configuración general de la aplicación:

```
C:\Users\Diego\Downloads\jhipster2025>jhipster
```



<https://www.jhipster.tech>

Welcome to JHipster v8.7.3

(node:18576) [DEP0040] DeprecationWarning: The `punycode` module is deprecated. Please use a userland alternative (Use `node --trace-deprecation ...` to show where the warning was created)

Documentation for creating an application is at <https://www.jhipster.tech/creating-an-app/>

Application files will be generated in folder: **C:\Users\Diego\Downloads\jhipster2025**

WARNING!

JHipster update available: **8.9.0** (current: 8.7.3)

Run **npm install -g generator-jhipster** to update.

```
✓ What is the base name of your application? jhipster2025
✓ Which *type* of application would you like to create? Monolithic application (recommended for simple projects)
✓ What is your default Java package name? com.udea
✓ Would you like to use Maven or Gradle for building the backend? Maven
✓ Do you want to make it reactive with Spring WebFlux? No
✓ Which *type* of authentication would you like to use? JWT authentication (stateless, with a token)
✓ Besides JUnit, which testing frameworks would you like to use?
✓ Which *type* of database would you like to use? SQL (H2, PostgreSQL, MySQL, MariaDB, Oracle, MSSQL)
✓ Which *production* database would you like to use? MySQL
✓ Which *development* database would you like to use? MySQL
✓ Which cache do you want to use? (Spring cache abstraction) Ehcache (local cache, for a single node)
✓ Do you want to use Hibernate 2nd level cache? Yes
✓ Which other technologies would you like to use?
✓ Which *framework* would you like to use for the client? React
✓ Besides Jest/Vitest, which testing frameworks would you like to use?
✓ Do you want to generate the admin UI? Yes
✓ Would you like to use a Bootswatch theme (https://bootswatch.com/)? Default JHipster
✓ Would you like to enable internationalization support? Yes
✓ Please choose the native language of the application Spanish
✓ Please choose additional languages to install English
```

Nota: Si usted ha escogido como framework de frontend a AngularJS es posible que deba ejecutar en secuencia los siguientes comandos, al finalizar la creación del proyecto:

- **npm install**
- **bower install**
- **gulp install**

Una vez finalizado la creación del proyecto podrá observar la siguiente salida:

Fundamentos Sistemas de Información

Universidad de Antioquia

```
npm WARN optional SKIPPING OPTIONAL DEPENDENCY: fsevents@^1.0.0 (node_modules\chokidar\node_modules\fsevents):
npm WARN notsup SKIPPING OPTIONAL DEPENDENCY: Unsupported platform for fsevents@1.1.1: wanted {"os":"darwin","arch":"any"} (current: {"os":"win32","arch":"x64"})
npm WARN @angular/compiler-cli@2.4.10 requires a peer of @angular/compiler@2.4.10 but none was installed.
npm WARN @angular/compiler-cli@2.4.10 requires a peer of @angular/core@2.4.10 but none was installed.

Server application generated successfully.

Run your Spring Boot application:
./mvnw

Client application generated successfully.

Start your Webpack development server with:
npm start
```

Nota: El tiempo de creación del proyecto puede tardar algunos minutos dependiendo de la velocidad de conexión a Internet.

Nota: La configuración general del proyecto está contenido en el archivo **.yo-rc.json** el cual tiene la siguiente estructura:

```
{
  "generator-jhipster": {
    "applicationType": "monolith",
    "authenticationType": "jwt",
    "baseName": "proyecto2025",
    "buildTool": "maven",
    "cacheProvider": "ehcache",
    "clientFramework": "react",
    "clientTestFrameworks": [],
    "clientTheme": "none",
    "creationTimestamp": 1742136714430,
    "databaseType": "sql",
    "devDatabaseType": "mysql",
    "devServerPort": 9060,
    "enableHibernateCache": true,
    "enableSwaggerCodegen": false,
    "enableTranslation": true,
    "entities": ["Contrato", "Departamento", "Direccion", "Empleado", "Pais", "Region", "Tarea", "Trabajo"],
    "feignClient": null,
    "jhipsterVersion": "8.7.3",
    "jwtSecretKey": "NDA5OWJiYWY0ZDNWRhYTEyODk0NGQ1NmVmZDliYjkwY2M=",
    "languages": ["es", "en"],
    "lastLiquibaseTimestamp": 1742139874000,
    "messageBroker": false,
    "microfrontend": null,
    "microfrontends": [],
    "nativeLanguage": "es",
    "packageName": "com.udea.proyecto",
    "prodDatabaseType": "mysql",
    "reactive": false,
    "searchEngine": false,
    "serverPort": null,
    "serverSideOptions": [],
    "serviceDiscoveryType": false,
    "syncUserWithIdp": null,
    "testFrameworks": [],
    "websocket": false,
    "withAdminUi": true
  }
}
```

Antes de proceder a construir el proyecto debe revisar la configuración de la base de datos, para lo cual diríjase a la siguiente ruta

C:\DirectorioProyectos\proyecto2025\src\main\resources\config y abra el archivo **application-dev.yml** y **application-prod.yml**.
 Revise en la sección **DataSource** que estén correctos los parámetros **URL**, **Username** y **Password**:

```
spring:
  devtools:
    restart:
      enabled: true
      additional-exclude: static/**
    liveload:
      enabled: false # we use Webpack dev server + BrowserSync for liveload
  jackson:
    serialization:
      indent-output: true
  datasource:
    type: com.zaxxer.hikari.HikariDataSource
    url: jdbc:mysql://localhost:3306/proyecto2025?useUnicode=true&characterEncoding=utf8
    username: root
    password: root
    hikari:
      poolName: Hikari
      auto-commit: false
      data-source-properties:
        cachePrepStmts: true
        prepStmtCacheSize: 250
        prepStmtCacheSqlLimit: 2048
        useServerPrepStmts: true
  liquibase:
    # Remove 'faker' if you do not want the sample data to be loaded automatically
    contexts: dev, faker
```

Una vez revisados los parámetros de conexión a la BD de MySQL, ahora ejecute el comando **mvnw** (en Windows) o **./mvnw** (En Linux). Este paso permitirá hacer un build al proyecto, cargar las dependencias necesarias y ejecutar el servidor web en el puerto 8080.

```
Administrator: Node.js command prompt - mvnw
C:\Users\Usuario\Documents\NetBeansProjects\blog1>mvnw
[INFO] Scanning for projects...
[INFO]
[INFO] -----
[INFO] Building Blog 1 0.0.1-SNAPSHOT
[INFO] -----
[INFO] >>> spring-boot-maven-plugin:1.4.4.RELEASE:run (default-cli) > test-compile @ blog-1 >>>
[INFO]
[INFO] --- maven-resources-plugin:3.0.1:copy-resources (default-resources) @ blog-1 ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 7 resources
[INFO] Copying 11 resources
[INFO]
[INFO] --- maven-resources-plugin:3.0.1:resources (default-resources) @ blog-1 ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 7 resources
[INFO] Copying 11 resources
[INFO]
[INFO] --- maven-enforcer-plugin:1.4.1:enforce (enforce-versions) @ blog-1 ---
[INFO]
[INFO] --- jacoco-maven-plugin:0.7.7.201606060606:prepare-agent (pre-unit-tests) @ blog-1 ---
[INFO] argLine set to -javaagent:C:\Users\Usuario\m2\repository\org\jacoco\org.jacoco.agent\destfile=C:\Users\Usuario\Documents\NetBeansProjects\blog1\target\test-results\coverage
[INFO]
[INFO] --- maven-compiler-plugin:3.6.0:compile (default-compile) @ blog-1 ---
[INFO] Changes detected - recompiling the module!
[INFO] Compiling 70 source files to C:\Users\Usuario\Documents\NetBeansProjects\blog1\target\classes
[INFO]
[INFO] --- maven-resources-plugin:3.0.1:testResources (default-testResources) @ blog-1 ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 3 resources
```

NOTA: Si hay algún problema en la ejecución inicial ejecute la siguiente secuencia de comandos:

- **npm install --save-dev ajv@^7**
- **npm run webapp:build:dev**
- **mvnw**

Nota: Debes tener activo el motor de Docker en su PC para que funcione.

Fundamentos Sistemas de Información

Universidad de Antioquia

```
Corresponding 'org.hibernate.cfg.AvailableSettings' constant for details.
2025-03-16T10:28:56.396-05:00 DEBUG 244 --- [cto-2025-task-1] t.j.c.Liquibase.AsyncSpringLiquibase : Liquibase has updated your database in 1423 ms
2025-03-16T10:28:58.511-05:00 DEBUG 244 --- [ restartedMain] c.a.JHipsterSpringDocGroupsConfiguration : Initializing JHipster OpenApi customizer
2025-03-16T10:28:58.523-05:00 DEBUG 244 --- [ restartedMain] c.a.JHipsterSpringDocGroupsConfiguration : Initializing JHipster OpenApi default group
2025-03-16T10:28:58.525-05:00 DEBUG 244 --- [ restartedMain] c.a.JHipsterSpringDocGroupsConfiguration : Initializing JHipster OpenApi management group
2025-03-16T10:28:58.757-05:00 DEBUG 244 --- [ restartedMain] i.m.c.u.i.logging.InternalLoggerFactory : Using SLF4J as the default logging framework
2025-03-16T10:28:59.154-05:00 INFO 244 --- [ restartedMain] org.jboss.threads : JBoss Threads version 3.5.0.Final
2025-03-16T10:28:59.215-05:00 INFO 244 --- [ restartedMain] com.udea.proyecto.Projecto2025App : Started Proyecto2025App in 20.851 seconds (process running for 21.259)
2025-03-16T10:28:59.221-05:00 INFO 244 --- [ restartedMain] com.udea.proyecto.Projecto2025App :

Application 'proyecto2025' is running! Access URLs:
Local:      http://localhost:8080/
External:   http://192.168.1.10:8080/
Profile(s): [dev, api-docs]

-----
2025-03-16T10:30:07.690-05:00 DEBUG 244 --- [ XNIO-1 task-2] c.u.p.web.rest.AuthenticateController : Enter: authorize() with argument[s] = [LoginVM(username='admin', rememberMe=false)]
2025-03-16T10:30:07.878-05:00 DEBUG 244 --- [ XNIO-1 task-2] c.u.p.security.DomainUserDetailsService : Authenticating admin
2025-03-16T10:30:08.129-05:00 DEBUG 244 --- [ XNIO-1 task-2] org.hibernate.SQL : select u1_0.id,u1_0.activated,u1_0.activation_key,a1_0.user_id,a1_1.name,u1_0.created_by,u1_0.email,u1_0.first_name,u1_0.image_url,u1_0.lang_key,u1_0.last_modified_by,u1_0.last_modified_date,u1_0.last_name,u1_0.login,u1_0.password_hash,u1_0.reset_date,u1_0.reset_key from jhi_user where u1_0.id=a1_0.user_id left join jhi_authority a1_1 on a1_1.name=a1_0.authority_name where u1_0.login=?
2025-03-16T10:30:08.430-05:00 DEBUG 244 --- [ XNIO-1 task-2] c.u.p.web.rest.AuthenticateController : Exit: authorize() with result = <200 OK OK,com.udea.proyecto.web.rest.AuthenticateCont
085a,[Authorization:Bearer eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJhZG1pb1IsImV4cCI6MTc0MjYlbnQwOCwiYXV0aCI6IjJPTeVfQUINSU4gUk9MRV9VU0V5IiwiaWF0IjoxNzQyMTM5MDA4fQ.1rT3qEq-H0Jcd1As4EQv6c5RehQ2zWE1v6k
usCb2Fw8KLMwA3G1hxvqBskpgxiawg"]>
2025-03-16T10:30:08.490-05:00 DEBUG 244 --- [ XNIO-1 task-2] c.u.proyecto.web.rest.AccountResource : Enter: getAccount() with argument[s] = []
2025-03-16T10:30:08.493-05:00 DEBUG 244 --- [ XNIO-1 task-2] com.udea.proyecto.service.UserService : Enter: getUserWithAuthorities() with argument[s] = []
2025-03-16T10:30:08.497-05:00 DEBUG 244 --- [ XNIO-1 task-2] com.udea.proyecto.service.UserService : Exit: getUserWithAuthorities() with result = Optional[User{login='admin', firstName='A
ame='Administrator', email='admin@localhost', imageUrl='', activated='true', langKey='es', activationKey='null'}]
2025-03-16T10:30:08.508-05:00 DEBUG 244 --- [ XNIO-1 task-2] c.u.proyecto.web.rest.AccountResource : Exit: getAccount() with result = AdminUserDTO{login='admin', firstName='Administrator
rator', email='admin@localhost', imageUrl='', activated=true, langKey='es', createdBy=system, createdAt=null, lastModifiedBy='system', lastModifiedDate=null, authorities=[ROLE_USER, ROLE_AD
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 02:30 min
[INFO] Finished at: 2025-03-16T10:30:45-05:00
[INFO]
```

Ahora en otra terminal ejecuta el comando **npm start**

```
<i> [webpack-dev-server] [HPM] Proxy created: /api,/services,/management,/v3/api-docs,/h2-console -> http://localhost:9000
Webpack: Starting ...

  ✓ Compile modules
  > Build modules (0%)
<i> [webpack-dev-server] Project is running at:
Webpack: Starting ...

  ✓ Compile modules
  ✓ Build modules
  ✓ Optimize modules
  ✓ Emit files

Finished after 12.877 seconds.

47 assets
645 modules
webpack 5.95.0 compiled successfully in 12841 ms
No errors found.
Notifications are disabled
Reason: DisabledForUser Please make sure that the app id is set correctly.
Command Line: C:\Users\Diego\Documents\proyecto2025\node_modules\node-notifier\node_modules\noretoast\noretoast-x64.exe -p
s\proyecto2025\webpack\logo-jhipster.png -m "Build successful" -t "Proyecto 2025"
[Browsersync] Proxying: http://localhost:9060
[Browsersync] Access URLs:
-----
    Local: http://localhost:9060
    External: http://192.168.1.10:9060
-----
    UI: http://localhost:3001
    UI External: http://localhost:3001
-----
(node:6420) [DEP0060] DeprecationWarning: The `util._extend` API is deprecated. Please use Object.assign() instead.
(Use `node --trace-deprecation ...` to show where the warning was created)
<i> [webpack-dev-server] Gracefully shutting down. To force exit, press ^C again. Please wait...
<i> [webpack-dev-server] [HPM] server close signal received: closing proxy server
Webpack: Starting ...

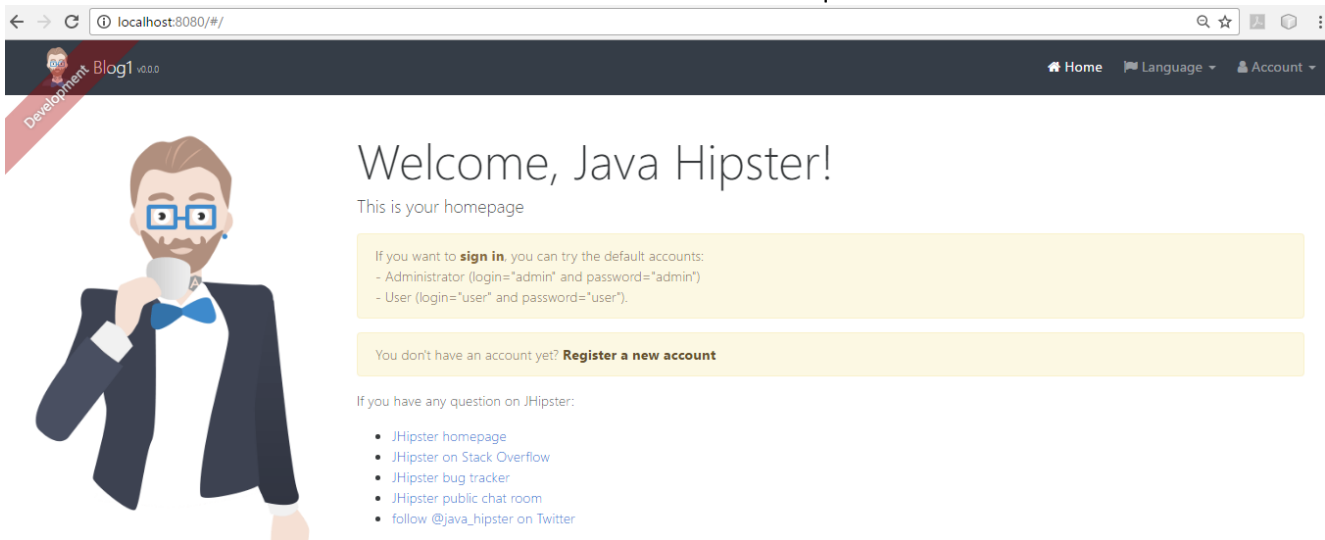
  ✓ Compile modules
  ✓ Build modules
  ✓ Optimize modules
  ✓ Emit files

Finished after 0.003 seconds.
```

Una vez listo el entorno de ejecución abra el navegador y coloque la siguiente dirección <http://localhost:8080>. Podrá observar la carga de la página como se observa a continuación. Pruebe ingresar con el usuario y contraseña admin o user configurados por defecto o crear nuevos usuarios. Puede ver también la sección de Internacionalización de la página.

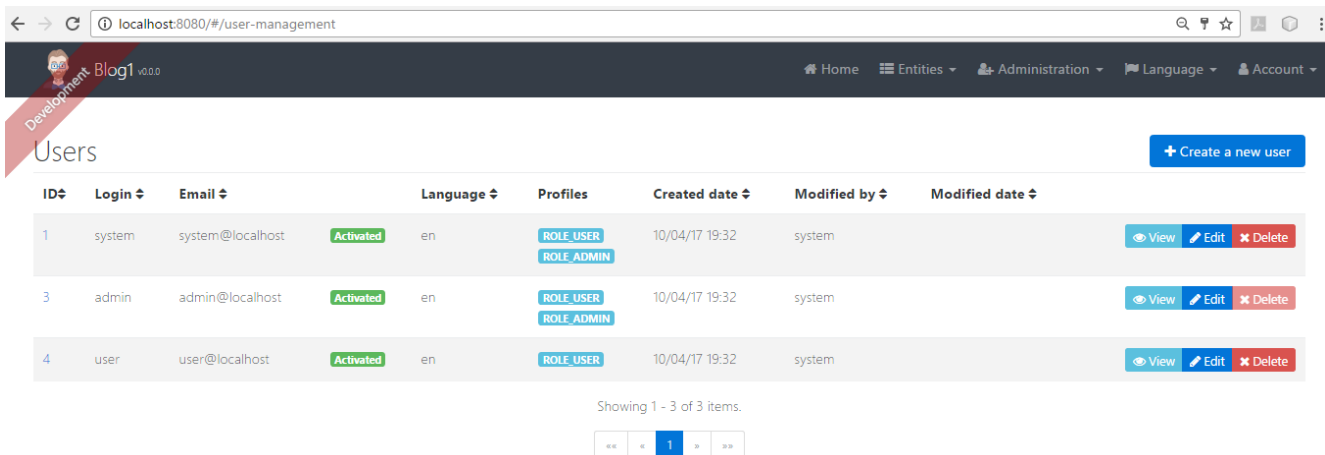
Fundamentos Sistemas de Información

Universidad de Antioquia



The screenshot shows the JHipster homepage. On the left is an illustration of a hipster with a beard and glasses, holding a coffee cup. The main heading is "Welcome, Java Hipster!" with the subtext "This is your homepage". Below this, there are two yellow boxes. The first box contains information about default accounts: "If you want to **sign in**, you can try the default accounts: - Administrator (login='admin' and password='admin') - User (login='user' and password='user')." The second box asks "You don't have an account yet? **Register a new account**". Below these boxes, there is a link "If you have any question on JHipster:" followed by a list of links: "JHipster homepage", "JHipster on Stack Overflow", "JHipster bug tracker", "JHipster public chat room", and "follow @java_hipster on Twitter". The browser's address bar shows "localhost:8080/#/" and the page title is "Blog1 v0.0.0".

Si usted se loguea como usuario **admin**, podrá acceder a la sección Administration, en donde se tendrá acceso a diversas opciones como monitoreo de la aplicación de Spring Boot, gestión de los usuarios, métricas de la aplicación, documentación de los servicios con Swagger, entre otros. A continuación, se presentan algunas de estas salidas.

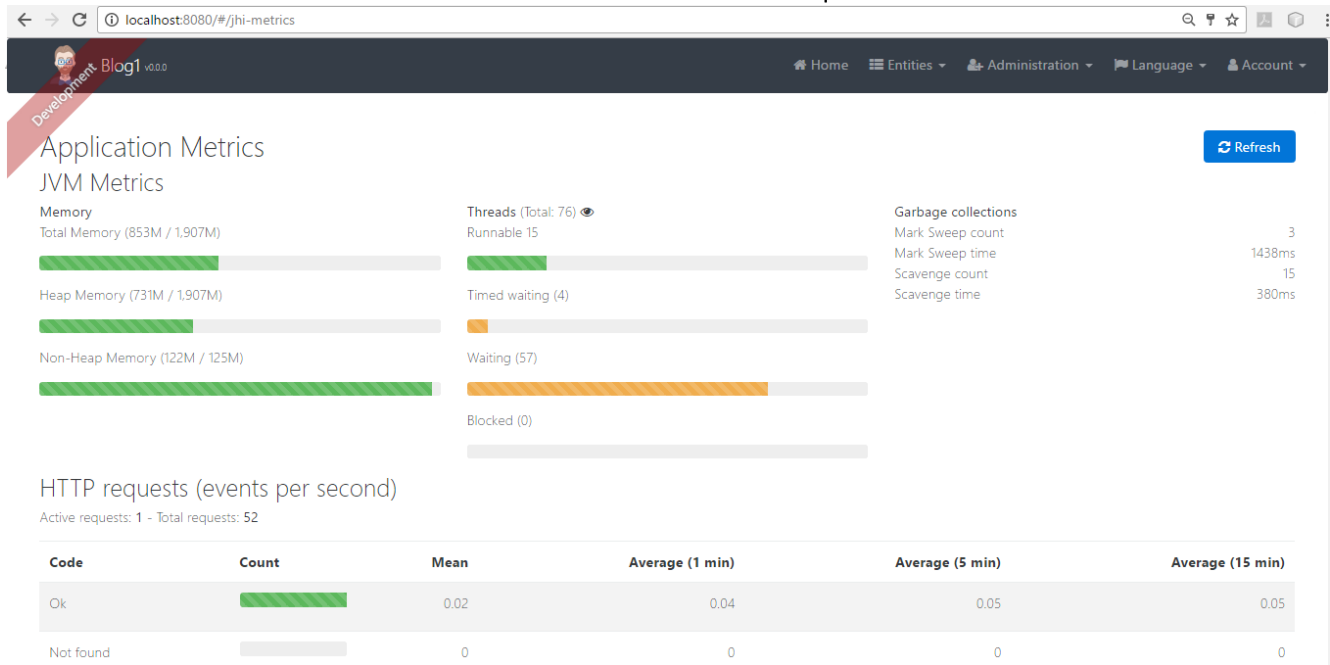


The screenshot shows the JHipster user management page. The browser's address bar shows "localhost:8080/#/user-management". The page title is "Blog1 v0.0.0". The navigation bar includes "Home", "Entities", "Administration", "Language", and "Account". The main heading is "Users" with a "+ Create a new user" button. Below the heading is a table with the following columns: ID, Login, Email, Language, Profiles, Created date, Modified by, and Modified date. The table contains three rows of user data. At the bottom, there is a pagination bar showing "Showing 1 - 3 of 3 items." and a page number "1".

ID	Login	Email	Language	Profiles	Created date	Modified by	Modified date	
1	system	system@localhost	Activated	en	ROLE_USER ROLE_ADMIN	10/04/17 19:32	system	View Edit Delete
3	admin	admin@localhost	Activated	en	ROLE_USER ROLE_ADMIN	10/04/17 19:32	system	View Edit Delete
4	user	user@localhost	Activated	en	ROLE_USER	10/04/17 19:32	system	View Edit Delete

Fundamentos Sistemas de Información

Universidad de Antioquia



localhost:8080/#/jhi-health

Blog1 v0.0.0

Health Checks

Service name	Status	Details
Disk space	UP	
Database	UP	

localhost:8080/#/docs

Blog1 v0.0.0

swagger

default (/v2/api-docs)

blog1 API

blog1 API documentation

account-resource : Account Resource

Show/Hide | List Operations | Expand Operations

GET	/api/account	getAccount
POST	/api/account	saveAccount
POST	/api/account/change_password	changePassword
POST	/api/account/reset_password/finish	finishPasswordReset
POST	/api/account/reset_password/init	requestPasswordReset
GET	/api/activate	activateAccount
GET	/api/authenticate	isAuthenticated
POST	/api/register	registerAccount

profile-info-resource : Profile Info Resource

Show/Hide | List Operations | Expand Operations

user-jwt-controller : User JWT Controller

Show/Hide | List Operations | Expand Operations

Fundamentos Sistemas de Información

Universidad de Antioquia

En este momento es buena idea iniciar el repositorio con Git colocando los siguientes comandos:

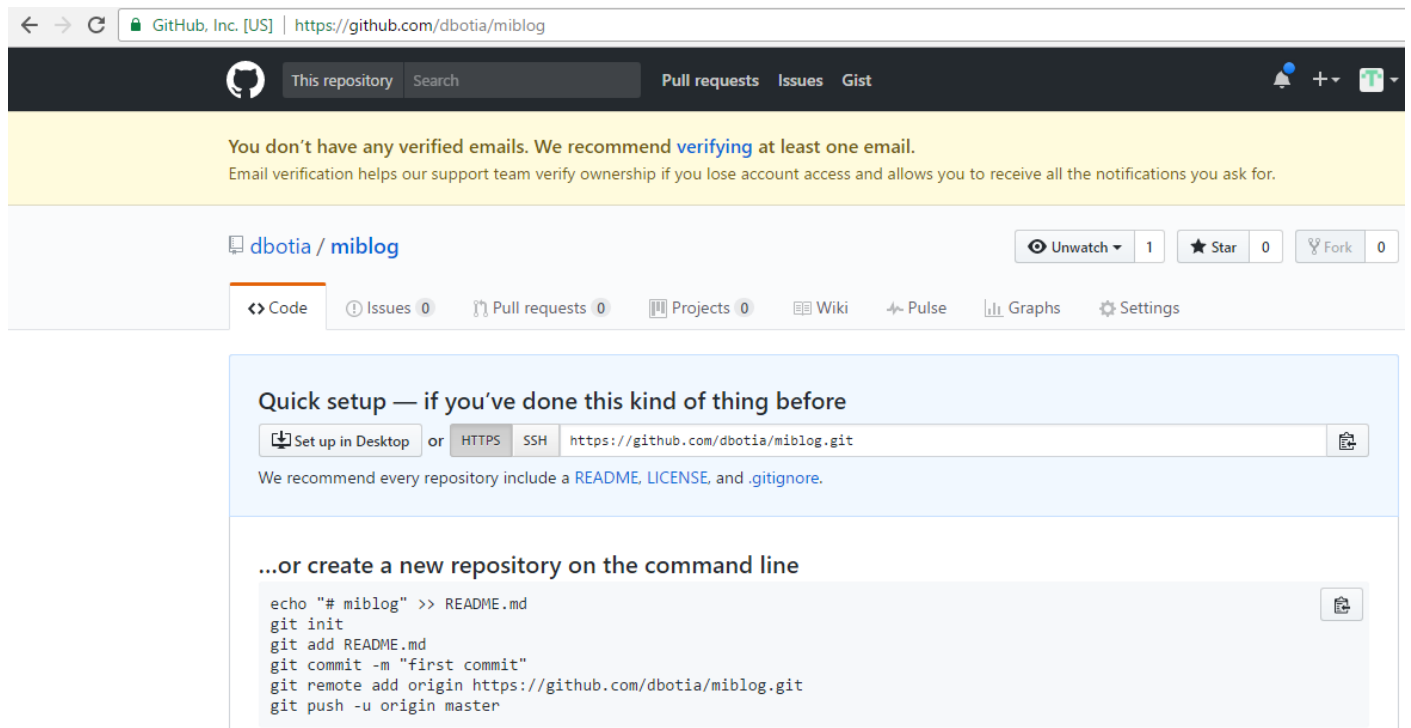
```
Administrator: Node.js command prompt

C:\Users\Usuario\Documents\NetBeansProjects\blog1>git init
Initialized empty Git repository in C:/Users/Usuario/Documents/NetBeansProjects/blog1/.git/

C:\Users\Usuario\Documents\NetBeansProjects\blog1>git add .

C:\Users\Usuario\Documents\NetBeansProjects\blog1>git commit -m "Proyecto Creado"
[master (root-commit) 1e921bd] Proyecto Creado
 318 files changed, 16716 insertions(+)
 create mode 100644 .editorconfig
 create mode 100644 .gitattributes
 create mode 100644 .gitignore
 create mode 100644 .mvn/wrapper/maven-wrapper.jar
 create mode 100644 .mvn/wrapper/maven-wrapper.properties
 create mode 100644 .travis.yml
 create mode 100644 .yo-rc.json
 create mode 100644 Jenkinsfile
 create mode 100644 README.md
 create mode 100644 angular-cli.json
 create mode 100644 mvnw
 create mode 100644 mvnw.cmd
 create mode 100644 package.json
 create mode 100644 pom.xml
 create mode 100644 postcss.config.js
 create mode 100644 src/main/docker/Dockerfile
 create mode 100644 src/main/docker/app.yml
 create mode 100644 src/main/docker/mysql.yml
 create mode 100644 src/main/docker/sonar.yml
 create mode 100644 src/main/java/com/udea/app/ApplicationWebXml.java
 create mode 100644 src/main/java/com/udea/app/Blog1App.java
```

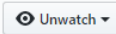
Una vez finalice el comando diríjase a su cuenta de github y cree un nuevo repositorio para asociarlo:

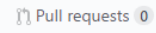
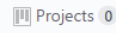


← → ↻  GitHub, Inc. [US] | <https://github.com/dbotia/miblog>

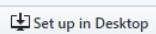
 This repository Search Pull requests Issues Gist   

You don't have any verified emails. We recommend [verifying](#) at least one email.
Email verification helps our support team verify ownership if you lose account access and allows you to receive all the notifications you ask for.

 dbotia / miblog  1  0  0

Quick setup — if you've done this kind of thing before

 or   

We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# miblog" >> README.md
git init
git add README.md
git commit -m "first commit"
git remote add origin https://github.com/dbotia/miblog.git
git push -u origin master
```

Fundamentos Sistemas de Información
Universidad de Antioquia

```
C:\Users\Usuario\Documents\miblog>git remote add origin https://github.com/dbotia/miblog.git

C:\Users\Usuario\Documents\miblog>git push -u origin master
Username for 'https://github.com': diego.botia@gmail.com
Password for 'https://diego.botia@gmail.com@github.com':
Counting objects: 441, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (419/419), done.
Writing objects: 100% (441/441), 273.84 KiB | 0 bytes/s, done.
Total 441 (delta 54), reused 0 (delta 0)
remote: Resolving deltas: 100% (54/54), done.
To https://github.com/dbotia/miblog.git
 * [new branch]      master -> master
Branch master set up to track remote branch master from origin.
```

GitHub, Inc. [US] | <https://github.com/dbotia/miblog>

dbotia / miblog

Unwatch 1 Star 0 Fork 0

Code Issues 0 Pull requests 0 Projects 0 Wiki Pulse Graphs Settings

No description, website, or topics provided. [Add topics](#) [Edit](#)

1 commit 1 branch 0 releases 1 contributor

Branch: master New pull request Create new file Upload files Find file Clone or download

dbotia Proyecto Nuevo		Latest commit ee79339 4 minutes ago
.mvn/wrapper	Proyecto Nuevo	3 minutes ago
src	Proyecto Nuevo	3 minutes ago
webpack	Proyecto Nuevo	3 minutes ago
.angular-cli.json	Proyecto Nuevo	3 minutes ago
.editorconfig	Proyecto Nuevo	3 minutes ago
.gitattributes	Proyecto Nuevo	3 minutes ago
.gitignore	Proyecto Nuevo	3 minutes ago
.yo-rc.json	Proyecto Nuevo	3 minutes ago

Generar las Entidades

Por cada entidad que se cree se necesitará lo siguiente:

- Una tabla en la base de datos.
- Un cambio dentro de Liquibase (www.liquibase.org), el cual es una librería para gestionar y monitorear los cambios en diferentes motores de base de datos.
- Una clase de tipo JPA Entity.
- Una interface JpaRepository con Spring Data.
- Una clase RestController con Spring MVC.
- Un modelo, estado, componente, componentes de diálogo, y servicios de Angular.
- Varias Páginas HTML5 por cada componente.

Además de lo anterior, se puede hacer test de integración para verificar el funcionamiento correcto de los componentes y test de rendimiento para verificar la rapidez de ejecución.

Fundamentos Sistemas de Información

Universidad de Antioquia

Para generar fácilmente todos estos componentes JHipster puede generar automáticamente cada uno de ellos, así como sus respectivas relaciones. Además, si usted genera entidades que se asocian a las vistas y controladores creados con Angular 5, todo el código HTML y TypeScript se creará también. También es posible realizar el proceso de validación de los atributos según sea el caso.

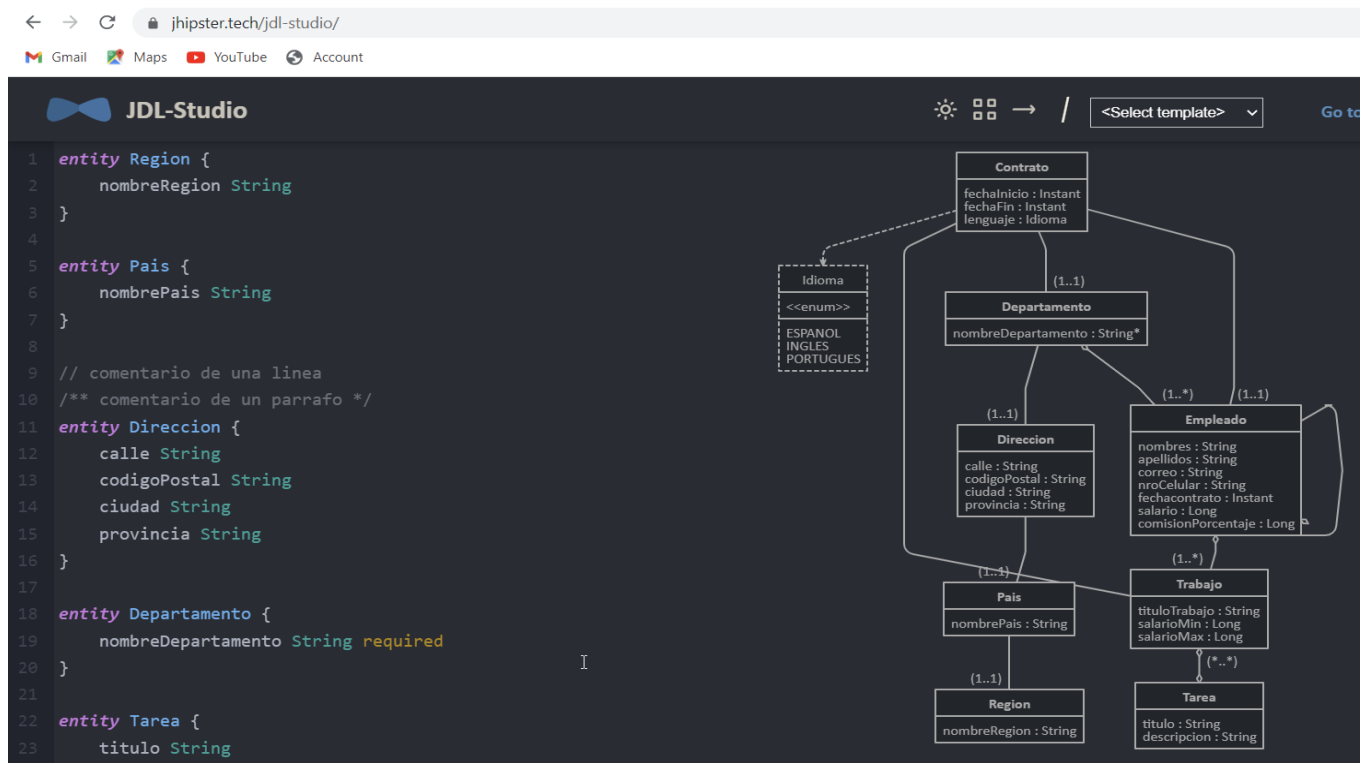
Generación del Código.

JHipster soporta varios métodos de generación de código. El primero es el sub generador de entidades el cual es una línea de comandos que por medio de preguntas facilita la creación de las entidades, atributos y relaciones de llaves foráneas. El problema es que puede ser un proceso tedioso según la cantidad de entidades que se generen. Para generar la entidad de esta forma escriba el comando **jhipster entity nombreEntidad**.

El siguiente método es usar la herramienta JHipster-UML que se integra con editores conocidos como Modelio (<https://www.modeliosoft.com/>), UML Designer (<http://www.uml designer.org/>), GenMyModel (<https://www.genmy model.com/>) y Visual Paradigm (<http://www.visual-paradigm.com/>).

El último método es a través de la herramienta JDL-Studio que es una herramienta web que permite definir el modelo del dominio con el lenguaje JDL (JHipster Domain Language). Este será la opción que se usará en este caso.

Primero desde su navegador abra la siguiente página <https://www.jhipster.tech/jdl-studio/>. Usted podrá observar un diagrama de entidades y código JDL.



Coloque el siguiente código en el editor para generar una simple aplicación web para gestionar y crear proyectos, el cual está formado por varias entidades.

```
entity Region {
  nombreRegion String
}

entity Pais {
  nombrePais String
}
```

Fundamentos Sistemas de Información
Universidad de Antioquia

```
// comentario de una linea
/** comentario de un parrafo */
entity Direccion {
    calle String
    codigoPostal String
    ciudad String
    provincia String
}

entity Departamento {
    nombreDepartamento String required
}

entity Tarea {
    titulo String
    descripcion String
}

entity Empleado {
    nombres String
    apellidos String
    correo String
    nroCelular String
    fechacontrato Instant
    salario Long
    comisionPorcentaje Long
}

entity Trabajo {
    tituloTrabajo String
    salarioMin Long
    salarioMax Long
}

entity Contrato {
    fechaInicio Instant
    fechaFin Instant
    lenguaje Idioma
}

enum Idioma {
    ESPANOL, INGLES, PORTUGUES
}

relationship OneToOne {
    Pais{region} to Region
}

relationship OneToOne {
    Direccion{pais} to Pais
}

relationship OneToOne {
    Departamento{direccion} to Direccion
}

relationship ManyToMany {
    Trabajo{tarea(titulo)} to Tarea{trabajo}
```

```
}

relationship OneToMany {
    Empleado to Trabajo{empleado}
    Departamento to Empleado{departamento}
}

relationship ManyToOne {
    Empleado{inmediatosuperior} to Empleado
}

// defining multiple oneToOne relationships
relationship OneToOne {
    Contrato{trabajo} to Trabajo
    Contrato{departamento} to Departamento
    Contrato{empleado} to Empleado
}

// mostrar con scroll
paginate Contrato, Empleado with infinite-scroll
paginate Trabajo with pagination

// Insertar opciones basicas a todos menos a Empleado, trabajo
service all with serviceImpl except Empleado, Trabajo
```

Ahora descargue el archivo desde el editor, desde el botón , el cual tendrá por defecto el nombre **jhipster-jdl.jdl**. (Si lo desea puede cambiarlo).

Para usar el generador de entidades con la versión 7 de Jhipster, debe ejecutar el comando **npm link generator-jhipster**. Ahora ejecute el comando **jhipster import-jdl jhipster-jdl.jdl** el cual importa el archivo JDL y genera las respectivas entidades, tests e interfaces gráficas con React.

Nota: Durante el proceso de creación de las entidades se le preguntará si desea sobrescribir el archivo **src/main/resources/config/liquibase/master.xml**, coloque la opción **a** y espere un momento.

Se recomienda ejecutar este comando 2 veces para que realice los cambios completamente.

Ahora en la ruta **\src\main\resources\config\application.yml** edítelo para agregar en la configuración de la sección JPA el dialecto de la Base de datos MySQL así:

```
jpa:
  open-in-view: false
  properties:
    hibernate.jdbc.time_zone: UTC
    hibernate.id.new_generator_mappings: true
    hibernate.connection.provider_disables_autocommit: true
    hibernate.cache.use_second_level_cache: true
    hibernate.cache.use_query_cache: false
    hibernate.generate_statistics: false
    hibernate.dialect: org.hibernate.dialect.MySQLDialect
    # modify batch size as necessary
    hibernate.jdbc.batch_size: 25
    hibernate.order_inserts: true
    hibernate.order_updates: true
    hibernate.query.fail_on_pagination_over_collection_fetch: true
    hibernate.query.in_clause_parameter_padding: true
```

Fundamentos Sistemas de Información
Universidad de Antioquia

Coloque de nuevo el comando **mvnw** y observe en el navegador que se han generado las nuevas vistas para las entidades. Intente ingresar datos de prueba en cada vista y vea su funcionalidad.

Nota: Para la prueba intente insertar datos usando los usuarios admin y users.

Si surge algún error ejecute los siguientes comandos **npm install react-jhipster** y **npm install react-router-dom**.

En el archivo **src/main/webapp/app/entities/menu.tsx** agregue los siguientes imports

```
import React from 'react';
import { MenuItem } from 'app/entities/menu-item';
import { Translate } from 'react-jhipster';
```

En el archivo **routes.tsx** de la misma carpeta agrega el siguiente import

```
import { Route } from 'react-router-dom';
```

En el archivo **menu-item.tsx** del mismo directorio agregar el siguiente export

```
import React from 'react';
import { DropdownItem } from 'reactstrap';
import { NavLink as Link } from 'react-router-dom';
import { FontAwesomeIcon } from '@fortawesome/react-fontawesome';
import { IconProp } from '@fortawesome/fontawesome-svg-core';

export interface IMenuItem {
  children: React.ReactNode;
  icon: IconProp;
  to: string;
  id?: string;
  'data-cy'?: string;
}

export const MenuItem = (props: IMenuItem) => {
  const { to, icon, id, children } = props;

  return (
    <DropdownItem tag={Link} to={to} id={id} data-cy={props['data-cy']}>
      <FontAwesomeIcon icon={icon} fixedWidth /> {children}
    </DropdownItem>
  );
};

export default MenuItem;
```

Ejecutar el comando **npm run lint --fix** y **npm start**

Fundamentos Sistemas de Información

Universidad de Antioquia

Proyecto20231 0.0.1 SNAPSHOT

Inicio Entidades Administración Español Cuenta

Entidades

- * Region
- * Pais
- * Direccion
- * Departamento
- * Tarea
- * Empleado
- * Trabajo
- * Contrato

Refrescar lista + Crear nuevo Tarea

ID	Titulo	Descripcion
1	global Account Central	quantifying Respuesta
2	1080p	bifurcada Rufiyaa invoice
3	Oficial Morado Fantástico	Lempira Canada HTTP
4	Soluciones Comunicaciones	streamline
5	navigating extend	Creativo Account
6	Montserrat paradigms International	Card deposit
7	de	acompasada Morado synergies
8	Checking programming Borders	calculating Movilidad Mascotas
9	program multitarea brand	Buckinghamshire
10	Granada redundant Azul	Galicia

Jhipster2025 vDEV

Inicio Entidades Administración Español Cuenta

Refrescar lista + Crear nuevo Trabajo

ID	Titulo Trabajo	Salario Min	Salario Max	Empleado
1	pro	31929	2717	
2	teriyaki hornet	14081	4240	
3	passionate through	9786	5316	
4	separately instance hmph	30159	21222	
5	sedately	4776	27834	
6	blah	25622	2412	
7	submitter peony majestically	3870	11499	
8	tapioca corrupt instead	5304	358	
9	tomorrow grounded nor	19259	31587	
10	sustenance hungrily after	23874	15706	

Ahora guarde los cambios de la lógica del negocio en Github. Coloque los siguientes comandos:

```
git add .
git commit -m "Cambios a la Logica del Negocio"
git push -u origin master
```

Usando Browsersync con webpack

BrowserSync es una herramienta de desarrollo web que permite sincronizar automáticamente los cambios realizados en el código fuente de un sitio web con múltiples navegadores en tiempo real.

Cuando se utiliza BrowserSync, cualquier cambio en el código, como la modificación de HTML, CSS o JavaScript, se refleja automáticamente en el navegador sin necesidad de actualizar la página manualmente. Esto es especialmente útil para desarrolladores web que trabajan en sitios web complejos y necesitan probar sus cambios en varios navegadores al mismo

Fundamentos Sistemas de Información
Universidad de Antioquia

tiempo.

Además de la sincronización en tiempo real, BrowserSync ofrece muchas otras funciones útiles, como la recarga automática de la página, la capacidad de navegar sincrónicamente en varios dispositivos y la posibilidad de inyectar CSS sin recargar la página. En resumen, BrowserSync es una herramienta valiosa para acelerar el proceso de desarrollo web y mejorar la eficiencia del trabajo.

Por otro lado Webpack es una herramienta de construcción de módulos para aplicaciones web. Permite empaquetar y optimizar múltiples módulos de JavaScript, CSS y otros recursos en un único archivo que se puede utilizar en un navegador web.

Para ejecutarlo escriba en una nueva terminal el comando **npm start** . Una vez finalice su ejecución deberá aparecer lo siguiente:

```
> proyecto-20231@0.0.1-SNAPSHOT webpack-dev-server
> webpack serve "--config" "webpack/webpack.dev.js" "--env" "stats=minimal"

<i> [webpack-dev-server] [HPM] Proxy created: /api,/services,/management,/v3/api-docs,/h2-console,/auth -> http://localhost:8080
Webpack: Starting ...

  ✓ Compile modules
  > Build modules (0%)
<i> [webpack-dev-server] Project is running at:
<i> [webpack-dev-server] Loopback: http://localhost:9060/
Webpack: Starting ...

  ✓ Compile modules
  ✓ Build modules
  ✓ Optimize modules
  ✓ Emit files

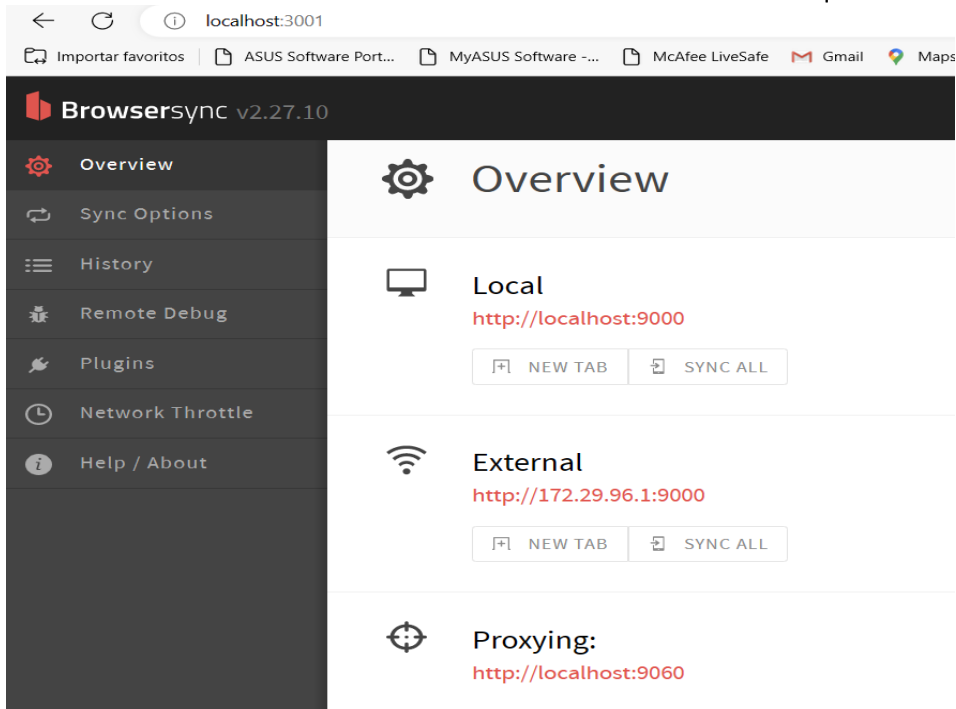
Finished after 3.101 seconds.

48 assets
674 modules
webpack 5.74.0 compiled successfully in 3084 ms
Type-checking in progress...
[Browsersync] Proxying: http://localhost:9060
[Browsersync] Access URLs:
  -----
    Local: http://localhost:9000
    External: http://172.29.96.1:9000
  -----
    UI: http://localhost:3001
    UI External: http://localhost:3001
  -----
No errors found.
```

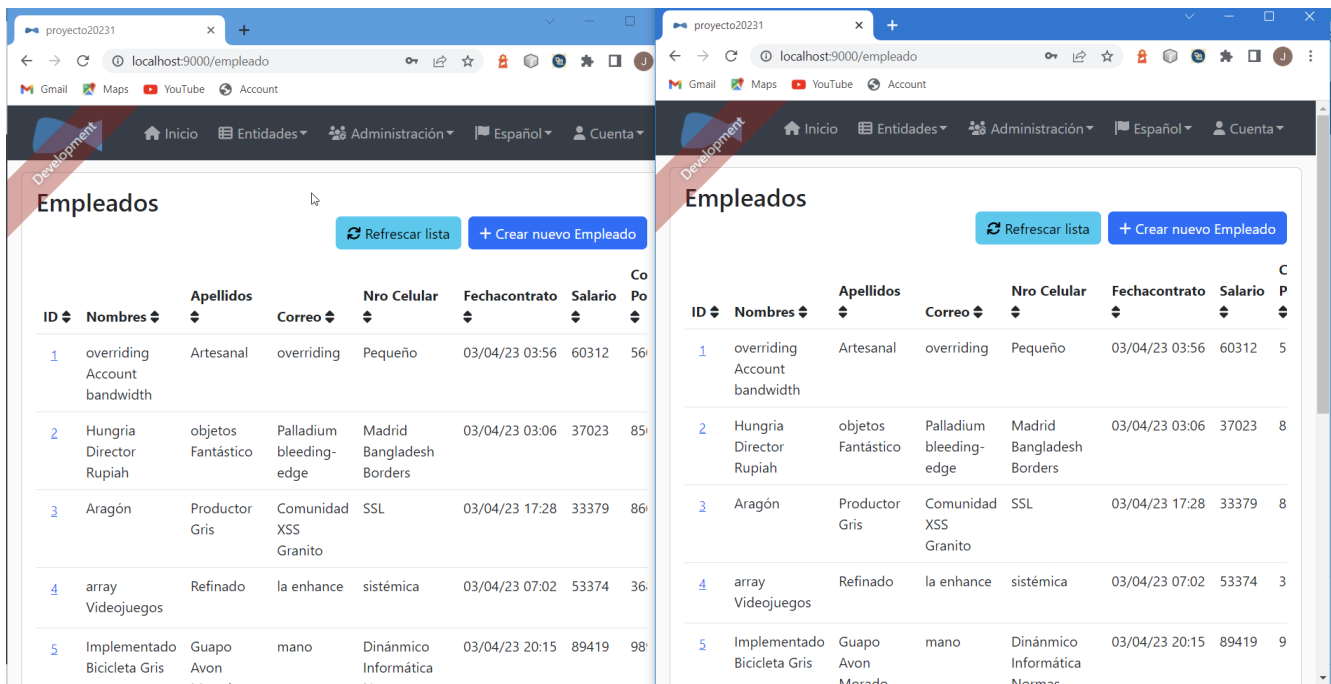
El acceso a la aplicación se hará por el puerto 9000 y el acceso al dashboard de Browsersync se hará en el puerto 3001.

Fundamentos Sistemas de Información

Universidad de Antioquia



Al abrir por ejemplo dos navegadores podrá ver como se sincronizan a cada acción que se genere.



Desplegar en ambiente de producción.

Cuando prepara la aplicación para el entorno de producción, se recomienda usar el perfil pre configurado **“prod”**. Con Maven usted puede empaquetar su aplicación para el perfil específico **“prod”** cuando se empieza a construir usando el comando **mvnw -Pprod package**. Si desea construir el proyecto y ejecutarlo escriba el comando **mvnw -Pprod**.

Nota: Tenga en cuenta que el proceso de construcción puede tardar varios minutos dependiendo de la cantidad de dependencias

a cargar.

```
Administrator: Node.js command prompt - mvnw -Pprod
C:\Users\Usuario\Documents\miblog>mvnw -Pprod
[INFO] Scanning for projects...
[INFO]
[INFO] -----
[INFO] Building Miblog 0.0.1-SNAPSHOT
[INFO] -----
[INFO]
[INFO] >>> spring-boot-maven-plugin:1.5.2.RELEASE:run (default-cli) > test-compile @ miblog >>>
[INFO]
[INFO] --- maven-resources-plugin:3.0.1:copy-resources (default-resources) @ miblog ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 11 resources
[INFO] Copying 11 resources
```

El perfil de producción es usado para construir un cliente optimizado de JavaScript. Usted puede invocarlo usando la herramienta webpack usando el comando **yarn run webpack:prod**. El perfil de producción también configura la compresión con GZIP con filtros servlets, cabeceras cache y monitoreo usando métricas.

Nota: Al finalizar el proceso observará la salida por consola como se muestra a continuación donde aparece cargado exitosamente el perfil “**prod**”. En este proceso también desde el directorio **target** se creará el respectivo archivo .war que facilitará su despliegue en cualquier servidor de aplicaciones.

```
: [{#transport#-1}{127.0.0.1}{127.0.0.1:9300}]
2017-04-15 12:01:40.120 ERROR 5448 --- [          main] .d.e.r.s.AbstractElasticsearchRepository : failed to load elast
icsearch nodes : org.elasticsearch.client.transport.NoNodeAvailableException: None of the configured nodes are available
: [{#transport#-1}{127.0.0.1}{127.0.0.1:9300}]
2017-04-15 12:01:47.049 INFO 5448 --- [          main] com.udea.app.MiblogApp                : Started MiblogApp in
42.993 seconds (JVM running for 44.672)
2017-04-15 12:01:47.052 INFO 5448 --- [          main] com.udea.app.MiblogApp                :
-----
Application 'miblog' is running! Access URLs:
Local:      http://localhost:8080
External:   http://192.168.1.69:8080
Profile(s): [prod]
-----
2017-04-15 12:12:01.387 DEBUG 5448 --- [ XNIO-2 task-9] i.g.j.s.Http401UnauthorizedEntryPoint : Pre-authenticated en
try point called. Rejecting access
```

Ejecución de Pruebas

JHipster viene con un extenso conjunto de pruebas, y cada aplicación generada tiene:

- Pruebas unitarias utilizando JUnit 5.
- Pruebas de integración con el framework Spring Test Context.
- Pruebas de interfaz de usuario con Jest.
- Pruebas de arquitectura con ArchUnit.

Opcionalmente, JHipster también puede generar:

- Pruebas de rendimiento con Gatling.
- Pruebas de comportamiento con Cucumber.
- Pruebas de integración Angular/React/Vue con Cypress o Protractor.

Lo primero será eliminar el directorio **timezone** que se encuentra en **src/test/java/com/xx/config**

Fundamentos Sistemas de Información

Universidad de Antioquia

nents\proyecto20231\src\test\java\com\udea\config

Nombre	Fecha de
timezone	3/04/2022
AsyncSyncConfiguration.java	3/04/2022
EmbeddedSQL.java	3/04/2022
MysqlTestContainer.java	3/04/2022
SpringBootTestClassOrderer.java	3/04/2022
SqlTestContainer.java	3/04/2022
StaticResourcesWebConfigurerTest.java	3/04/2022
TestContainersSpringContextCustomizerF...	3/04/2022
WebConfigurerTest.java	3/04/2022
WebConfigurerTestController.java	3/04/2022

Para ejecutar la batería de pruebas puede usar el comando **mvnw clean verify**. El realizará toda la ejecución de pruebas durante unos minutos y si todo sale bien arrojará la siguiente salida

```
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.732 s - in com.udea.web.rest.errors.ExceptionTranslatorIT
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 204, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco-maven-plugin:0.8.8:report-integration (post-integration-tests) @ proyecto-20231 ---
[INFO] Loading execution data file C:\Users\Diego\Documents\proyecto20231\target\jacoco-it.exec
[INFO] Analyzed bundle 'Proyecto 20231' with 90 classes
[INFO]
[INFO] --- maven-checkstyle-plugin:3.1.2:check (default) @ proyecto-20231 ---
[INFO] You have 0 Checkstyle violations.
[INFO]
[INFO] --- maven-failsafe-plugin:3.0.0-M7:verify (verify) @ proyecto-20231 ---
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 04:44 min
[INFO] Finished at: 2023-04-05T16:29:53-05:00
[INFO]
```

Si quiere ver el resultado de cada test puede ir al siguiente directorio **\target\failsafe-reports**

Para hacer pruebas de integración y cobertura con el frontend usaremos el comando **npm test** que nos arrojará un resultado similar al siguiente:

```
Test Suites: 25 passed, 25 total
Tests:       266 passed, 266 total
Snapshots:   0 total
Time:        18.452 s, estimated 35 s
Ran all test suites.
```

Nota: para mejorar las pruebas resultantes instale los siguientes módulos **npm i @testing-library/react** y **npm i @testing-library/dom**

Pruebas de Stress con Gatling

Para generar pruebas de estrés para ver el performance del Sistema, podemos emplear la herramienta Gatling que generará datos de prueba para verificar la respuesta de nuestra aplicación. Para hacerlo descargue la herramienta Gatling de su página oficial (<http://gatling.io>) y desde la consola de comando ejecute lo siguiente:

Fundamentos Sistemas de Información
Universidad de Antioquia

```
C:\Users\Diego\Downloads\lab1fsi>
C:\Users\Diego\Downloads\lab1fsi>mvnw gatling:test -Dgatling.simulationClass=gatling.simulations.ContratoGatlingTest
[INFO] Scanning for projects...
[INFO] -----< com.udea:lab-1-fsi >-----
[INFO] Building Lab 1 Fsi 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] >>> gatling:4.10.1:test (default-cli) > test-compile @ lab-1-fsi >>>
[INFO] --- enforcer:3.5.0:enforce (enforce-versions) @ lab-1-fsi ---
[INFO] Rule 0: org.apache.maven.enforcer.rules.version.RequireMavenVersion passed
[INFO] Rule 1: org.apache.maven.enforcer.rules.version.RequireJavaVersion passed
[INFO] --- enforcer:3.5.0:enforce (enforce-dependencyConvergence) @ lab-1-fsi ---
[INFO] Rule 0: org.apache.maven.enforcer.rules.dependency.DependencyConvergence passed
[INFO] --- properties:1.2.1:read-project-properties (default) @ lab-1-fsi ---
[INFO] Loading 36 properties from File: C:\Users\Diego\Downloads\lab1fsi\sonar-project.properties
[INFO] --- jacoco:0.8.12:prepare-agent (pre-unit-tests) @ lab-1-fsi ---
[INFO] argLine set to -javaagent:C:\Users\Diego\m2\repository\org\jacoco\org.jacoco.agent\0.8.12\org.jacoco.agent-0.8.12-runtime.jar -Djava.security.egd=file:/dev/./urandom -Xmx1G
[INFO] --- spotless:2.43.0:apply (spotless) @ lab-1-fsi ---
[INFO] Spotless.Java is keeping 189 files clean - 0 were changed to be clean, 0 were already clean, 189 were skipped because cache
[INFO] --- spring-boot:3.3.5:build-info (build-info) @ lab-1-fsi ---
[INFO] --- checksum:1.11:files (create-pre-compiled-webapp-checksum) @ lab-1-fsi ---
[INFO] --- antrun:3.1.0:run (eval-frontend-checksum) @ lab-1-fsi ---
[INFO] Executing tasks
[INFO] Executed tasks
[INFO] --- frontend:1.15.1:install-node-and-npm (install-node-and-npm) @ lab-1-fsi ---
[INFO] Node v22.11.0 is already installed.
[INFO] NPM 10.9.0 is already installed.
```

Nota: puede colocar el llamado a las clases que desee y que estén dentro del archivo de simulaciones.

Para ejecutar las pruebas observe que en su proyecto este creado el archivo `..\src\test\gatling\simulations\*GatlingTest.java`. Estos archivos contienen los parámetros de la prueba de carga para gatling. El contenido del archivo es el siguiente:

```
/**
 * Performance test for the Contrato entity.
 *
 * @see <a href="https://github.com/jhipster/generator-jhipster/tree/v8.7.3/generators/gatling#logging-tips">Logging tips</a>
 */
public class ContratoGatlingTest extends Simulation {

    String baseUrl =
Optional.ofNullable(System.getProperty("baseUrl")).orElse("http://localhost:8080");

    HttpProtocolBuilder httpConf = http
        .baseUrl(baseUrl)
        .inferHtmlResources()
        .acceptHeader("*/*")
        .acceptEncodingHeader("gzip, deflate")
        .acceptLanguageHeader("fr,fr-fr;q=0.8,en-us;q=0.5,en;q=0.3")
        .connectionHeader("keep-alive")
        .userAgentHeader("Mozilla/5.0 (Macintosh; Intel Mac OS X 10.10; rv:33.0)
Gecko/20100101 Firefox/33.0")
```

Fundamentos Sistemas de Información

Universidad de Antioquia

```
.silentResources(); // Silence all resources like css or css so they don't
clutter the results

Map<String, String> headersHttp = Map.of("Accept", "application/json");

Map<String, String> headersHttpAuthentication = Map.of("Content-Type",
"application/json", "Accept", "application/json");

Map<String, String> headersHttpAuthenticated = Map.of("Accept", "application/json",
"Authorization", "${access_token}");

ChainBuilder scn = exec(http("First unauthenticated
request").get("/api/account").headers(headersHttp).check(status().is(401)))
    .exitHereIfFailed()
    .pause(10)
    .exec(
        http("Authentication")
            .post("/api/authenticate")
            .headers(headersHttpAuthentication)
            .body(StringBody("{\"username\":\"admin\", \"password\":\"admin\"}"))
            .asJson()
            .check(header("Authorization").saveAs("access_token"))
        )
    .exitHereIfFailed()
    .pause(2)
    .exec(http("Authenticated
request").get("/api/account").headers(headersHttpAuthenticated).check(status().is(200)))
    .pause(10)
    .repeat(2)
    .on(
        exec(http("Get all
contratoes").get("/api/contratoes").headers(headersHttpAuthenticated).check(status().is(
200)))
            .pause(Duration.ofSeconds(10), Duration.ofSeconds(20))
            .exec(
                http("Create new contrato")
                    .post("/api/contratoes")
                    .headers(headersHttpAuthenticated)
                    .body(
                        StringBody(
                            "{" +
                            "\"fechaInicio\": \"2020-01-01T00:00:00.000Z\" +
                            "\", \"fechaFin\": \"2020-01-01T00:00:00.000Z\" +
                            "\", \"lenguaje\": \"ESPAÑOL\" +
                            "}"
                        )
                    )
                )
            .asJson()
            .check(status().is(201))
            .check(headerRegex("Location",
"(.*)").saveAs("new_contrato_url"))
        )
    .exitHereIfFailed()
    .pause(10)
    .repeat(5)
    .on(exec(http("Get created
contrato").get("${new_contrato_url}").headers(headersHttpAuthenticated)).pause(10))
    .exec(http("Delete created
contrato").delete("${new_contrato_url}").headers(headersHttpAuthenticated))
```

Fundamentos Sistemas de Información
Universidad de Antioquia

```
        .pause(10)
    );

ScenarioBuilder users = scenario("Test the Contrato entity").exec(scen);

{
    setUp(
        users.injectOpen(rampUsers(Integer.getInteger("users",
100)).during(Duration.ofMinutes(Integer.getInteger("ramp", 1))))
    ).protocols(httpConf);
}
}
```

Estas simulaciones pueden tomar unos minutos en completarse.

Al finalizar debe aparecer el resumen de las ejecuciones de las simulaciones donde se observa las llamadas exitosas y no exitosas de cada API así:

```
2025-09-18T15:05:22.251-05:00 INFO --- [      main] io.gatling.app.Runner           : Simulation gatling.simulations.ContratoGatlingTest complete
Parsing log file(s)...
2025-09-18T15:05:22.293-05:00 INFO --- [      main] io.gatling.charts.stats.LogFileReader$ : Collected C:\Users\Diego\Downloads\lab1fsi\target\gatling\c
og from contratogatlingtest-20250918200340779
2025-09-18T15:05:22.308-05:00 INFO --- [      main] io.gatling.charts.stats.LogFileReader   : First pass
2025-09-18T15:05:22.319-05:00 INFO --- [      main] io.gatling.charts.stats.LogFileReader   : First pass done: read 701 lines
2025-09-18T15:05:22.338-05:00 INFO --- [      main] io.gatling.charts.stats.LogFileReader   : Second pass
2025-09-18T15:05:22.436-05:00 INFO --- [      main] io.gatling.charts.stats.LogFileReader   : Second pass: read 701 lines
Parsing log file(s) done in 0s.
Generating reports...

=====
--- Global Information -----
> request count                500 (OK=200   KO=300 )
> min response time            2 (OK=3      KO=2   )
> max response time            1066 (OK=1066  KO=7   )
> mean response time           31 (OK=72     KO=3   )
> std deviation                101 (OK=151   KO=1   )
> response time 50th percentile 4 (OK=84    KO=3   )
> response time 75th percentile 7 (OK=88    KO=4   )
> response time 95th percentile 91 (OK=98   KO=5   )
> response time 99th percentile 585 (OK=993  KO=6   )
> mean requests/sec            4.95 (OK=1.98  KO=2.97 )
--- Response Time Distribution -----
> t < 800 ms                   196 ( 39.2%)
> 800 ms <= t < 1200 ms        4 ( 0.8%)
> t >= 1200 ms                  0 ( 0%)
> failed                        300 ( 60%)
--- Errors -----
> status.find.is(200), but actually found 401      200 (66,67%)
> status.find.is(201), but actually found 401      100 (33,33%)
=====

Reports generated, please open the following file: file:///C:/Users/Diego/Downloads/lab1fsi/target/gatling/contratogatlingtest-20250918200340779/index.html
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 02:30 min
[INFO] Finished at: 2025-09-18T15:05:23-05:00
[INFO] -----
```

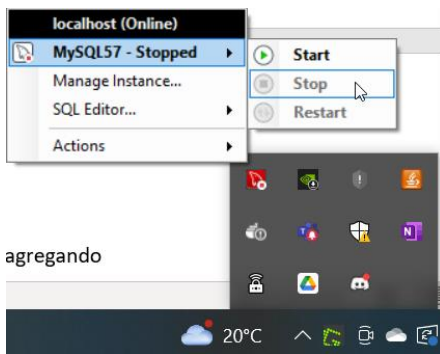
Cuando finalice podrá observar reportes interactivos como se muestra a continuación dentro de la carpeta de reportes de gatling.



Despliegue con Docker

El despliegue es bastante sencillo: JHipster genera un archivo yaml que puede llamarse a Docker Compose. Basado en ese archivo, se inician dos contenedores - uno para una base de datos MySQL y un segundo para la aplicación (que contiene el backend Java basado en Spring y maneja las peticiones HTTP desde el navegador que ejecuta la también incluida aplicación web React y utiliza Hibernate y una conexión JDBC para hablar con la base de datos en el primer contenedor).

Lo primero será parar la ejecución local del servidor MySQL. Por lo tanto, el proceso lo detenemos para que se libere el puerto 3306.



Ahora configuramos el archivo `..src\main\docker\app.yml` editando las siguientes líneas:

```
SPRING_DATASOURCE_URL=jdbc:mysql://proyecto2025-  
mysql:3306/proyecto2025?useUnicode=true&characterEncoding=utf8&allowPublicKeyRetrieval=  
true&useSSL=false&useLegacyDatetimeCode=false&serverTimezone=UTC&createDatabaseIfNotExist  
=true
```

```
- SPRING_LIQUIBASE_URL=jdbc:mysql://proyecto2025-  
mysql:3306/proyecto2025?useUnicode=true&characterEncoding=utf8&allowPublicKeyRetrieval=  
true&useSSL=false&useLegacyDatetimeCode=false&serverTimezone=UTC&createDatabaseIfNotExist  
=true
```

Ahora cambiamos la propiedad - `SPRING_PROFILES_ACTIVE=dev`, `api-docs`

Fundamentos Sistemas de Información

Universidad de Antioquia

Ahora agregamos la siguiente configuración al archivo `..src\main\docker\mysql.yml` así:

```
environment:
  - MYSQL_ALLOW_EMPTY_PASSWORD=yes
  - MYSQL_DATABASE=proyecto2025
  - MYSQL_ROOT_HOST=proyecto2025-mysql
  - MYSQL_ROOT_PASSWORD=root
  - MYSQL_USER=root
  - MYSQL_PASSWORD=root
```

Ahora creamos el archivo **Dockerfile** en la misma carpeta con el siguiente contenido:

```
FROM openjdk:11-jre-slim-stretch

ENV SPRING_OUTPUT_ANSI_ENABLED=ALWAYS \
    JHIPSTER_SLEEP=0 \
    JAVA_OPTS=""

RUN adduser -D -s /bin/sh jhipster
WORKDIR /home/jhipster

ADD entrypoint.sh entrypoint.sh
RUN chmod 755 entrypoint.sh && chown jhipster:jhipster entrypoint.sh
USER jhipster

ENTRYPOINT ["/entrypoint.sh"]

EXPOSE 8080

ADD *.war app.war
```

Usaremos ahora el comando **jhipster docker-compose** para activar la imagen que generaremos por medio de Docker. El creará el archivo **docker-compose.yml** en la raíz del proyecto.

Fundamentos Sistemas de Información

Universidad de Antioquia

```
C:\Users\Diego\Documents\proyecto20231>jhipster docker-compose
[INFO] Switching to JHipster installed locally in current project's node repository (node:20936) [DEP0148] DeprecationWarning: Use of deprecated folder mapping ".lib/util" in "package.json"
environment/package.json.
Update this package.json to use a subpath pattern like ".lib/util/*".
(Use `node --trace-deprecation ...` to show where the warning was created)

JHIPSTER
https://www.jhipster.tech

Welcome to JHipster v7.9.3

✓ Docker is installed

Found .yo-rc.json config file...
? Which *type* of application would you like to deploy? Monolithic application
? Enter the root directory where your applications are located ../
3 applications found at C:\Users\Diego\Documents\

? Which applications do you want to include in your configuration? proyecto20231
? Do you want to setup monitoring for your applications ? No
Welcome to the JHipster Docker Compose Sub-Generator
Files will be generated in folder: C:\Users\Diego\Documents\proyecto20231

Checking Docker images in applications directories...
ls: no such file or directory: C:/Users/Diego/Documents/proyecto20231/target/jib-cache
force .yo-rc-global.json
force .yo-rc.json
force ..\proyecto2023\..yo-rc.json
create docker-compose.yml
create README-DOCKER-COMPOSE.md
force ..\proyectojh\..yo-rc.json

No change to package.json was detected. No package manager install will be executed.

WARNING! Docker Compose configuration generated, but no Jib cache found
If you forgot to generate the Docker image for this application, please run:
To generate the missing Docker image(s), please run:
./mvnw -ntp -Pprod verify jib:dockerBuild in C:\Users\Diego\Documents\proyecto20231

You can launch all your infrastructure by running : docker-compose up -d
Congratulations, JHipster execution is complete!
Sponsored with by @oktadev.
```

Los enfoques de compilación de Maven, Gradle y NPM para JHipster son extensos y están bien documentados. Pueden utilizarse fácilmente para producir una imagen de contenedor Docker aprovechando el proceso local del demonio Docker. (Ver <https://www.jhipster.tech/docker-compose/#3>).

JHipster utiliza la biblioteca Jib que ayuda a crear imágenes de contenedores para aplicaciones Java. El proceso de construcción NPM no puede trabajar con jib de esta manera, pero tanto Maven y Gradle sí.

Para generar la imagen ejecutamos el comando **mvnw package -Pdev verify jib:dockerBuild**

```
[INFO] Containerizing application to Docker daemon as proyecto20231...
[WARNING] Base image 'eclipse-temurin:11-jre-focal' does not use a specific image digest - build may not be reproducible
[INFO] The base image requires auth. Trying again for eclipse-temurin:11-jre-focal...
[WARNING] The credential helper (docker-credential-desktop) has nothing for server URL: registry-1.docker.io
[WARNING]
Got output:

credentials not found in native keychain

[WARNING] The credential helper (docker-credential-desktop) has nothing for server URL: registry.hub.docker.com
[WARNING]
Got output:

credentials not found in native keychain

[INFO] Using credentials from Docker config (C:\Users\Diego\.docker\config.json) for eclipse-temurin:11-jre-focal
[INFO] Using base image with digest: sha256:b2609d27aeec66b36976822bea9139e296cda399c2fbb8bdce2d8e1ffe563170
[INFO]
[INFO] Container entrypoint set to [bash, -c, /entrypoint.sh]
[INFO]
[INFO] Built image to Docker daemon as proyecto20231
[INFO] Executing tasks:
[INFO] [=====] 100,0% complete
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 03:04 min
[INFO] Finished at: 2023-04-05T22:53:44-05:00
[INFO] -----
```

Una vez que la imagen del contenedor de la aplicación se ha construido exitosamente, ejecutamos la aplicación con el siguiente

Fundamentos Sistemas de Información

Universidad de Antioquia

comando.

```
docker-compose -f src/main/docker/app.yml up -d
```

Si la imagen se despliega exitosamente en el contenedor, podrá observar la siguiente salida:

```
proyecto20231-app_1 | 2023-04-06T05:00:55.200Z WARN 1 --
hibernate.exception.GenericJDBCException: Unable to acquire
proyecto20231-app_1 | 2023-04-06T05:00:55.202Z WARN 1 --
tion: Could not open JPA EntityManager for transaction; nest
proyecto20231-app_1 | 2023-04-06T05:00:55.216Z WARN 1 --
proyecto20231-app_1 | 2023-04-06T05:00:55.219Z WARN 1 --
n: Full authentication is required to access this resource]
Gracefully stopping... (press Ctrl+C again to force)
Stopping docker_proyecto20231-mysql_1 ... done
Stopping docker_proyecto20231-app_1 ... done
```

Para verificar la ejecución exitosa del contenedor y la exposición de sus puertos puede usar el comando **docker container ls**

```
C:\Users\Diego\Documents\proyecto20231>docker container ls
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
aae28441fee0   mysql:8.0.30   "docker-entrypoint.s..." 12 hours ago   Up About a minute   127.0.0.1:3306->3306/tcp, 33060/tcp   docker_proyecto20231-mysql_1
b4b5cff7df2c   proyecto20231 "bash -c /entrypoint..." 12 hours ago   Up About a minute   127.0.0.1:8080->8080/tcp              docker_proyecto20231-app_1
```

Si tiene instalado Docker Desktop puede verificarlo como se muestra a continuación, en la sección Containers:

proyecto20231 b4b5cff7df2c (docker_proyecto20231-app_1)	4 minutes ago	running	    
mysql:8.0.30 aae28441fee0 (docker_proyecto20231-mysql_1)	4 minutes ago	running	    

Si usted ejecuta la aplicación desde el contenedor recientemente creado, observará que, si se intenta acceder con algunas de las cuentas predeterminadas, genera un error de acceso. Si usted analiza la traza del log de salida de la aplicación podrá mirar que la BD esta apuntando a la IP 172.18.0.X la cual no tiene configurado ningún password.

docker_proyecto20231-app_1 proyecto20231
RUNNING

Logs Inspect Stats

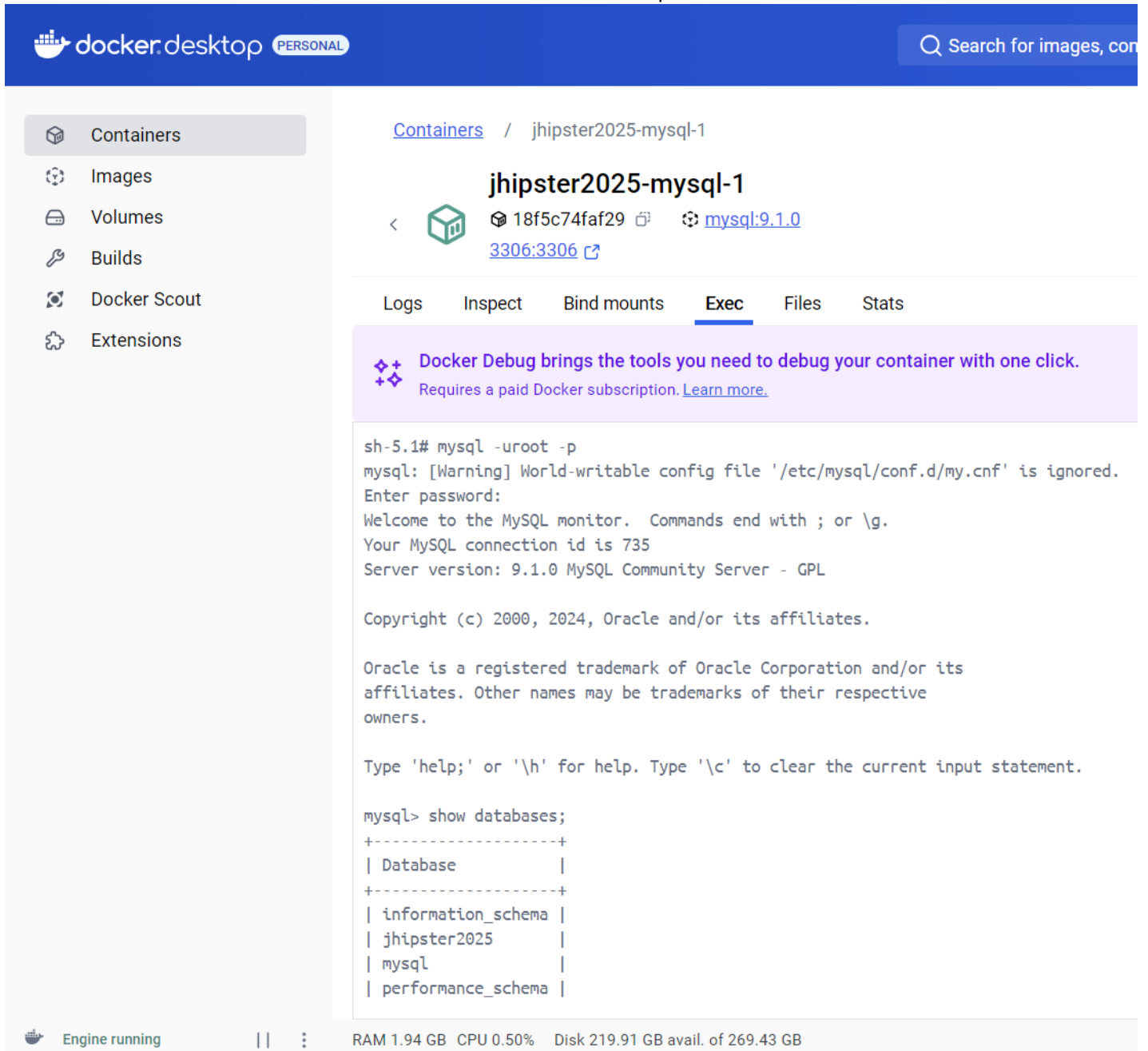
```
at com.udea.Proyecto20231App.main(Proyecto20231App.java:70)
at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(Unknown Source)
at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(Unknown Source)
at java.base/java.lang.reflect.Method.invoke(Unknown Source)
at org.springframework.boot.devtools.restart.RestartLauncher.run(RestartLauncher.java:49)

2023-04-06T17:09:00.592Z WARN 1 --- [ restartedMain] o.h.e.j.e.i.JdbcEnvironmentInitiator : HHH000342: Could not obtain connection to query metadata

java.sql.SQLException: Access denied for user 'root'@'172.18.0.3' (using password: YES)
at com.mysql.cj.jdbc.exceptions.SQLException.createSQLException(SQLException.java:129)
at com.mysql.cj.jdbc.exceptions.SQLExceptionsMapping.translateException(SQLExceptionsMapping.java:122)
```

Para solucionar este problema, abrimos el contenedor de MySQL y abrimos una consola.





The screenshot shows the Docker Desktop interface. On the left is a sidebar with navigation options: Containers, Images, Volumes, Builds, Docker Scout, and Extensions. The main area displays the details for a container named 'jhipster2025-mysql-1'. The 'Exec' tab is selected, showing a terminal window. The terminal output shows the MySQL command prompt and the output of the 'show databases;' command.

Containers / jhipster2025-mysql-1

jhipster2025-mysql-1

18f5c74faf29 mysql:9.1.0 3306:3306

Logs Inspect Bind mounts **Exec** Files Stats

Docker Debug brings the tools you need to debug your container with one click. Requires a paid Docker subscription. [Learn more.](#)

```
sh-5.1# mysql -uroot -p
mysql: [Warning] World-writable config file '/etc/mysql/conf.d/my.cnf' is ignored.
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 735
Server version: 9.1.0 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+-----+
| Database                |
+-----+
| information_schema       |
| jhipster2025             |
| mysql                    |
| performance_schema       |
+-----+
```

Engine running | RAM 1.94 GB CPU 0.50% Disk 219.91 GB avail. of 269.43 GB

En la nueva consola ingresaremos con el usuario root (sin password) a mysql y luego creamos un nuevo usuario root a la IP asignada por Docker y le asignaremos una contraseña.

Fundamentos Sistemas de Información

Universidad de Antioquia

```
docker exec -it aae28441fee0873e4a9d2e19f22ac2cc4419eb87fc6793ebd5a038bef7b35a1f /bin/sh
sh-4.4# mysql -uroot -p
mysql: [Warning] World-writable config file '/etc/mysql/conf.d/my.cnf' is ignored.
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.30 MySQL Community Server - GPL

Copyright (c) 2000, 2022, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE USER 'root'@'172.18.0.2' IDENTIFIED BY 'root';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT ALL PRIVILEGES ON *.* TO 'root'@'172.18.0.2';
Query OK, 0 rows affected (0.01 sec)
```

Ahora reiniciamos los contenedores de Docker y ya podremos acceder sin problemas a nuestra aplicación.

The screenshot shows a web browser at localhost:8080/empleado. The application header includes 'Proyecto20231 0.0.1 SNAPSHOT' and navigation links: Inicio, Entidades, Administración, Español, and Cuenta. The main section is titled 'Empleados' and features a table with employee data. Above the table are buttons for 'Refrescar lista' and '+ Crear nuevo Empleado'. Each row in the table has 'Vista', 'Editar', and 'Eliminar' buttons.

ID	Nombres	Apellidos	Correo	Nro Celular	Fechacontrato	Salario	Comision Porcentaje	Inmediatosuperior	Departamento
1	overriding Account bandwidth	Artesanal	overriding	Pequeño	02/04/23 22:56	60312	56071		
2	Hungria Director Rupiah	objetos Fantástico	Palladium bleeding-edge	Madrid Bangladesh Borders	02/04/23 22:06	37023	85068		
3	Aragón	Productor Gris	Comunidad XSS Granito	SSL	03/04/23 12:28	33379	86035		
4	array Videojuegos	Refinado	la enhance	sistémica	03/04/23 02:02	53374	36812		
5	Implementado Bicicleta Gris	Guapo Avon Morado	mano	Dinámico Informática Normas	03/04/23 15:15	89419	98928		
6	Producto copying	Rioja deliver Barbados	holística Avon	Agente Algodón dynamic	03/04/23 15:34	34218	25126		

Usando TLS y HTTP/2

JHipster soporta el protocolo de seguridad TLS y HTTP/2 en desarrollo (principalmente para pruebas).

TLS es el protocolo utilizado cuando se tiene una URL <https://>, y es necesario para utilizar HTTP/2 en los navegadores modernos.

Es útil emplear estos protocolos cuando se prueba una aplicación, principalmente por razones de rendimiento.

JHipster tiene una configuración específica para configurar tanto TLS como HTTP/2, y para simplificar aún más las cosas:

- JHipster genera un certificado autofirmado en la generación de la aplicación
- Se proporciona un perfil `tls` específico

Para ejecutar JHipster con el certificado autofirmado proporcionado, con TLS y HTTP/2 habilitados, necesitas usar el perfil `tls`:

- Con Maven: `./mvnw -Pdev,tls`

Fundamentos Sistemas de Información

Universidad de Antioquia

La aplicación estará disponible en <https://localhost:8080/>.

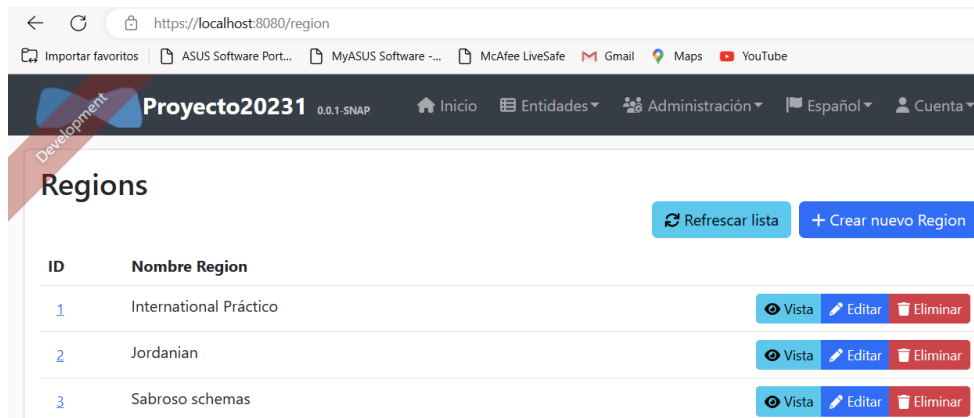
Como el certificado es autofirmado, tu navegador emitirá un aviso, y tendrás que ignorarlo (o importarlo) para poder acceder a la aplicación

La salida deberá ser similar a la siguiente donde se observa el despliegue con el protocolo https del dominio local.

```
2023-04-06T17:45:26.581-05:00 INFO 22244 --- [ restartedMain] com.udea.Proyecto20231App
2023-04-06T17:45:26.588-05:00 INFO 22244 --- [ restartedMain] com.udea.Proyecto20231App

-----
Application 'proyecto20231' is running! Access URLs:
Local:      https://localhost:8080/
External:   https://192.168.1.9:8080/
Profile(s): [dev, api-docs, tls]
```

Para ejecutar con React ejecute el comando **npm run start-tls**



Ejercicio propuesto. Entrega: 23 de septiembre de 2026

De acuerdo al siguiente archivo JDL que representa un sistema de una aerolínea virtual, cree una aplicación con JHipster y despliéguela en docker. (Nota la base de datos debe ser MySQL). En el informe de laboratorio presente el respectivo proceso.

Si desea puede ampliar o completar las entidades.

```
application {
  config {
    baseName aerolineaVirtual
    applicationType monolith
    databaseType sql
    devDatabaseType mysql
    prodDatabaseType mysql
    buildTool maven
  }
}

entity Pasajero {
  nombre String required
  apellido String required
  email String required unique
  telefono String
  fechaNacimiento LocalDate
}

entity Reserva {
  codigo String required unique
  fechaReserva Instant required
  estado String required
}
```

Fundamentos Sistemas de Información
Universidad de Antioquia

```
}

entity Vuelo {
  numeroVuelo String required unique
  origen String required
  destino String required
  fechaSalida Instant required
  fechaLlegada Instant required
}

entity Asiento {
  numero String required
  clase String required
  disponible Boolean required
}

relationship OneToMany {
  Pasajero to Reserva
}

relationship ManyToOne {
  Reserva to Vuelo
  Reserva to Asiento
}

paginate Pasajero, Reserva, Vuelo, Asiento with pagination

service Pasajero, Reserva, Vuelo, Asiento with serviceClass
```

